

M2Lean: A Certificate-Based Bridge Between Macaulay2 and Lean 4

Andrew Tawfeek*

July 2026

Abstract

Computer algebra systems and proof assistants occupy separate ecosystems: *Macaulay2* discovers and computes at scale but its answers carry no formal guarantee, while *Lean* certifies mathematics but lacks serious computational commutative algebra. We present *M2Lean*, a bridge built on a simple contract: *Macaulay2* is an untrusted oracle that must justify its answers with *certificates*, and *Lean* re-verifies those certificates through small checkers backed by kernel-checked soundness theorems. The bridge consists of a versioned mathematical interchange protocol, a *Macaulay2* package that exports computations together with the evidence needed to reproduce their correctness (cofactors, change matrices, and division data realizing Buchberger’s criterion), and a *Lean 4* library that parses, validates, checks, and—for the checkers whose soundness is formalized—produces genuine **mathlib** theorems. We distinguish explicitly between *checked* computations and *proved* ones, and we demonstrate the full path on a flagship example: *Macaulay2* discovers that the cone over the twisted cubic and a line in $\mathbb{A}^4$ are disjoint and emits a Nullstellensatz-style certificate; *Lean* re-checks the certificate inside its kernel and concludes—with no new axioms and no **sorry**—that the two varieties have no common point over *any* nontrivial commutative $\mathbb{Q}$ -algebra, a statement *Macaulay2* itself cannot even formulate. Corrupted certificates, including ones obtained by mutating real *Macaulay2* output, are rejected. We describe the architecture, the trust model, what each system stands to gain, and the road from this vertical slice to a certified computational commutative algebra.

1 Introduction

A working algebraic geometer who wants to *compute*—a Gröbner basis, a free resolution, a primary decomposition—reaches for a computer algebra system such as *Macaulay2* [GS24]. A mathematician who wants to *certify*—to produce a machine-checked proof—reaches for a proof assistant such as *Lean 4* [dMU21] with its mathematical library **mathlib** [The20]. These two activities ought to be two halves of one workflow. In practice they are disconnected: *Macaulay2*’s answers are the output of three decades of highly optimized, unverified C++ and interpreted code, and *Lean*’s algebra library, while remarkably broad, contains almost none of the executable machinery of computational commutative algebra.

M2Lean is a bridge between the two systems designed around a division of labor rather than a translation:

*University of Washington. coffeeandproofs@gmail.com. The implementation described here was developed with the assistance of Claude (Anthropic). Code and all artifacts: <https://github.com/andrew-tawfeek/M2Lean>.

<i>Macaulay2</i>	the <i>computational laboratory</i> : discovers examples, computes bases, resolutions, invariants, decompositions
<i>Lean</i>	the <i>logical authority</i> : fixes the mathematical meaning of those computations and certifies conclusions

The goal is not to translate *Macaulay2* source code into *Lean*, nor to verify *Macaulay2*’s implementation. It is to create a shared mathematical language and a bidirectional protocol through which a computation can cross from one system to the other *without losing its meaning, its assumptions, or its checkability*.

The skeptic’s contract. M2Lean adopts the classical skeptical stance toward computer algebra [HT98]: *Macaulay2* is treated as an untrusted oracle. A result is never accepted because *Macaulay2* printed `true`; it is accepted only when *Macaulay2* supplies *evidence*—cofactors expressing an element in terms of generators, change-of-basis data, division quotients for S-polynomials—and a small *Lean* checker verifies that evidence. For the core checkers we prove soundness theorems in *Lean*: if the checker accepts, a genuine `mathlib` proposition holds. A defect anywhere in *Macaulay2*, in the bridge code, or in the serialization can cause spurious *rejection*, but never a false theorem.

Contributions. This paper describes the design and a complete working vertical slice of the bridge, the *M2Lean Bridge Core*:

1. **A versioned interchange protocol** (§4) for exact commutative algebra over $\mathbb{Q}$ and prime fields: polynomial rings with normatively specified monomial orders, ideals, matrices, graded free modules, and six kinds of *claims with evidence*. Canonical forms are strict enough that equality of encodings is structural, and consumers reject rather than repair.
2. **A *Macaulay2* bridge package** (§5) that exports objects and produces certificates from public *Macaulay2* interfaces, including Gröbner certificates whose S-pair evidence is generated by an explicit division algorithm so that the standard-representation bounds of Buchberger’s criterion hold literally.
3. **A *Lean 4* library** (§6) with an executable sparse-polynomial model, a parser and strict validator for the protocol, certificate checkers, and machine-checked soundness theorems connecting checker acceptance to propositions about `MvPolynomial (Fin n) ℚ` in `mathlib`: polynomial identity, ideal membership, span inclusion and equality, unit ideals, and vanishing composition of matrices. A command-line verifier `m2lean-check` reports per-claim verdicts.
4. **Explicit assurance levels** (§3). “Computed,” “checked,” and “proved” are distinct statuses that the system reports and never conflates. The Gröbner checker, for example, verifies Buchberger’s criterion computationally but its abstract soundness theorem is future formalization work; it therefore reports `checked`, and no *Lean* theorem in the repository consumes it.
5. **A flagship theorem** (§7) exercising the entire path: from an *Macaulay2* discovery, through a certificate, through kernel-checked verification, to an abstract geometric conclusion in *Lean* that neither system could produce alone—with an axiom audit showing no additions to *Lean*’s standard three axioms.

6. **Adversarial testing** (§8): mutated real certificates (a altered S-pair quotient, a deleted S-pair, a corrupted differential entry, wrong cofactors) and malformed or non-canonical documents are all rejected, each with a diagnostic that distinguishes parse, structural, and verification failure.

Everything is open source; the repository contains the protocol specification, both implementations, all fixtures, and scripts that reproduce every claim in this paper [Taw26].

2 Related work

Skeptical combination of proof assistants and computer algebra goes back to Harrison and Théry’s HOL–Maple bridge [HT98], which introduced the pattern we follow: let the CAS find an object that is expensive to discover but cheap to check (a factorization, an antiderivative), and re-verify it in the prover. Delahaye and Mayero connected Coq to Maple for field simplification [DM05], and Kaliszyk and Wiedijk prototyped a full CAS on top of HOL Light [KW07]. Within *Lean*, the `linear_combination` tactic [Gol22] checks exactly the kind of cofactor certificate our membership checker consumes, and `polyrith` [Bha22] obtains those cofactors by calling an external Gröbner engine—a special case of the M2Lean pattern, built directly into a tactic. M2Lean generalizes this from single tactic calls to a versioned, provenance-carrying protocol spanning objects (rings, ideals, matrices, complexes) and claim families, with an explicitly stratified trust model.

On the formalization side, Buchberger’s algorithm was verified in Coq by Théry [Thé01] and Gröbner theory is developed in Isabelle/HOL [IM16]; `mathlib` currently has no comparable executable development, which is precisely the gap the `checked` tier of M2Lean is designed to close from the outside in. CoqEAL [CDM13] refines abstract algebraic structures to efficient executable ones inside the prover; M2Lean is complementary, keeping the heavy computation outside the prover and importing only evidence. Certificate-based cooperation is also standard practice in SMT proof reconstruction [BBFF20] and in sums-of-squares certificates for nonlinear arithmetic [Har07]; M2Lean can be read as bringing that discipline to computational commutative algebra, where the natural certificates are cofactors, syzygies, and division data [CLO15, BW93].

3 Architecture and trust model

3.1 Four layers

layer	realization
interchange representation	<code>protocol/SPEC.md</code> , JSON schema, fixtures
semantics	<i>Lean</i> : <code>sparse model</code> $\rightarrow$ <code>MvPolynomial (Fin n) ℚ</code>
claims and certificates	<i>Lean</i> checkers + soundness theorems
user-facing integrations	<i>Macaulay2</i> package; <code>m2lean-check</code> CLI

A document travels left to right: *Macaulay2* exports objects and claims, *Lean* parses and *validates* (arities, canonical scalars, dependency order, dimension checks—rejecting, never repairing), then runs one checker per claim and emits a verification report. Parsing is not proof: the semantics layer is what turns byte-level checks into mathematics.

3.2 Trusted and untrusted

The trusted base consists of the *Lean* kernel, the definitions giving protocol objects mathematical meaning (the sparse model and its interpretation), the statements of the soundness theorems, and **mathlib** itself. Explicitly untrusted: all of *Macaulay2* (engine, interpreter, our package), all serialization and transport, every field of every document including provenance, and generated *Lean* syntax prior to kernel checking. Checkers may not use **native_decide**; certificate checks in theorem files run through the kernel (**decide +kernel**).

3.3 Assurance levels

Every accepted claim carries one of four levels, and the distinction is preserved end to end—in the checker code, in the CLI report, and in which claims example theorems are permitted to consume:

transported	object parsed and validated; nothing checked
reproduced	computation re-run and matched; no certificate
checked	certificate verified by an executable checker whose mathematical soundness is cited but not yet formalized
proved	as checked , plus a kernel-checked soundness theorem: acceptance <i>implies</i> the mathlib proposition

In the current implementation, polynomial identity, ideal membership, span inclusion, and chain-complex composition are **proved**; Gröbner bases and graded complexes are **checked**. We regard surfacing this distinction as a design contribution rather than a confession: systems that blur “the checker ran” and “the theorem holds” invite exactly the silent failures a bridge like this exists to eliminate.

4 The interchange protocol

Version 0.1.0 of the protocol deliberately covers a small, exactly specified fragment: coefficients in $\mathbb{Q}$ (reduced fractions of decimal strings; JSON numbers are never used for mathematical data) or $\mathbb{F}_p$ (p verified prime by the consumer); multivariate polynomial rings with positional variable identity and one of two normatively defined monomial orders (**Lex** and **GRevLex**, *Macaulay2*’s default); sparse polynomials in canonical form (nonzero reduced coefficients, exact arity, strictly decreasing terms); ideals; matrices; and graded free modules $\bigoplus_j R(-d_j)$.

Canonicity is strict enough that equality of encoded polynomials is structural equality, which is the base of every checker, and it is enforced, not assumed: a document whose terms are misordered for the *declared* order is rejected. This addresses a classical source of silent cross-system corruption—graded reverse lexicographic order alone admits several incompatible textbook conventions—by making the definitions normative in the specification and testing both implementations against shared adversarial fixtures ordered differently under **Lex** and **GRevLex**.

A *claim* is a proposition about previously introduced objects plus evidence addressed to a specific checker:

claim kind	proposition	evidence
PolynomialIdentity	$lhs = rhs$	none (normalization)
IdealMembership	$f \in (g_1, \dots, g_k)$	cofactors c_i with $f = \sum c_i g_i$
SpanInclusion	$I \subseteq J$	a cofactor row per generator
GroebnerBasis	G is a GB of I	span witnesses both ways + S-pair standard representations
ChainComplex	$d_i \circ d_{i+1} = 0$	none (matrix arithmetic)
GradedComplex	the above, graded	none (homogeneity checks)

Documents also carry provenance—producer versions, algorithm names, determinism flags—which is displayed for reproducibility but never influences acceptance. The verifier answers with a report document whose failure messages distinguish parse, structural, and verification failures.

The most interesting evidence format is the Gröbner one. For every pair $i < j$ of basis elements whose leading monomials are not coprime, the producer must supply quotients $q_1, \dots, q_r$ such that

$$S(b_i, b_j) = \sum_k q_k b_k, \quad \text{lm}(q_k b_k) \preceq \text{lm}(S(b_i, b_j)) \text{ whenever } q_k \neq 0,$$

i.e. a *standard representation*. By Buchberger’s criterion in standard-representation form [BW93, Thm. 5.64] [Buc70, CLO15], verifying these identities and bounds (plus the two span inclusions relating G to the input generators) establishes that G is a Gröbner basis. The point of this format is that the checker never runs Buchberger’s algorithm: it only checks polynomial identities and monomial comparisons, both of which reduce to the structural equality of canonical forms.

5 The Macaulay2 bridge package

The package `m2/M2Lean.m2` exports objects and generates certificates using public *Macaulay2* interfaces. Because everything it produces is re-verified, it may freely use *Macaulay2*’s untrusted machinery: membership cofactors come from matrix division `f // gens I` against *Macaulay2*’s internal Gröbner engine, and basis conversion witnesses likewise. The one place where honesty about *how* a result was obtained matters is the S-pair evidence: *Macaulay2*’s engine does not expose its reduction traces, so the package re-derives the quotients with an explicit textbook division algorithm (twenty lines of *Macaulay2* code) whose loop invariant guarantees the standard-representation bound that the protocol demands. This is a small instance of a phenomenon we expect to recur: *certificate export forces implicit algorithmic guarantees to become explicit*, and occasionally forces recomputation in a shape that carries its own correctness evidence.

One subtlety deserves record. *Macaulay2* keeps polynomials sorted in the ring’s monomial order, so canonical export is free—but only after the bridge verifies that *Macaulay2*’s order conventions agree with the protocol’s normative definitions. Our test suite pins this down empirically (e.g. $x + y^2z$ must serialize as $[x, y^2z]$ under `Lex` and $[y^2z, x]$ under `GRevLex`), and the shared fixtures would catch any drift on either side.

6 The Lean library

6.1 An executable model and its interpretation

The computational heart is a deliberately small sparse model:

```

structure STerm where
  coeff : Rat
  exs : List Nat -- positional; entries beyond the end are 0
abbrev SPoly := List STerm

```

with executable normalization (sort by a fixed total comparator, then merge equal exponent vectors and drop zeros, with trailing-zero trimming so that paddings of the same monomial coincide), ring operations on raw term lists, and the two protocol monomial orders. All recursion is structural, for a reason that becomes visible later: well-founded recursion does not reduce definitionally, and these functions must run *inside the kernel*.

The semantics is a function `toMv n : SPoly → MvPolynomial (Fin n) ℚ` interpreting a term list as a sum of monomials, together with a lemma suite proved once and for all: `toMv` commutes with concatenation, term-wise product, permutation, merging, and normalization. The payoff is the soundness of the two executable predicates every checker is built from:

```

theorem polyEq_sound (h : polyEq p q = true) : toMv n p = toMv n q
theorem polyIsZero_sound (h : polyIsZero p = true) : toMv n p = 0

```

6.2 Checkers and soundness theorems

Checkers are Boolean functions on plain sparse data—independent of the protocol AST, so the soundness theorems apply to them directly. Representative statements (all `sorry`-free, all checked by the kernel):

```

theorem checkMembership_sound
  (h : checkMembership f gs cs = true) :
  toMv n f ∈ Ideal.span {p | p ∈ gs.map (toMv n)}

theorem checkSpanInclusion_sound
  (h : checkSpanInclusion src tgt rows = true) :
  spanOf n src ≤ spanOf n tgt

theorem checkComplexPair_sound
  (h : checkComplexPair r m c A B = true) :
  toMatrix n r m A * toMatrix n m c B = 0

theorem no_common_zero
  (h : checkMembership onePoly gs cs = true)
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra ℚK]
  (pt : Fin 4 → K)
  (hv : ∀ g ∈ gs, aeval pt (toMv n g) = 0) : False

```

The last theorem is the abstract half of the flagship: a certificate that $1 = \sum c_i g_i$ makes the span the unit ideal, and the unit ideal has no common zero in any nontrivial commutative $\mathbb{Q}$ -algebra—the certificate refutes points valued in every field extension at once, which is exactly the content usually delivered by the Nullstellensatz but obtained here by pure algebra from an explicit witness.

The Gröbner checker `checkGroebner` verifies non-coprime S-pair coverage, each standard representation with its leading-monomial side conditions, and both span inclusions. It is complete

and executable, but the implication “criterion verified $\Rightarrow$ Gröbner basis” is Buchberger’s theorem, which we have not yet formalized; the claim therefore reports **checked** and no example theorem consumes it. Formalizing the criterion against this checker—and upstreaming the reusable parts (monomial orders, standard representations, division) to **mathlib**—is the most valuable single step on the roadmap, and would promote every existing Gröbner certificate to **proved** without touching *Macaulay2* or the protocol.

6.3 Kernel-checked certificate consumption

The CLI accepts documents at runtime, but example *theorems* embed certificate data as literals (generated deterministically from the JSON by a checked-in script) and verify them by kernel reduction:

```
theorem certificate_checks :
  checkMembership onePoly gens cofactors = true := by decide +kernel
```

Two implementation notes may save others time. First, any well-founded recursion in a checker silently blocks such kernel evaluation; our merge step is a structural **foldr** for exactly this reason. Second, **Rat** arithmetic does not reduce in the elaborator’s evaluator but does in the kernel (which has fast arbitrary-precision arithmetic including **Nat.gcd**), so **decide +kernel** rather than plain **decide** is required.

7 The flagship example

The example is chosen to be mathematically recognizable, small enough for continuous integration, and structured so that both systems are essential (cf. [Eis95]). Let $R = \mathbb{Q}[x, y, z, w]$ and let

$$I = (xz - y^2, xw - yz, yw - z^2)$$

be the ideal of 2×2 minors of $\begin{pmatrix} x & y & z \\ y & z & w \end{pmatrix}$, cutting out the cone over the twisted cubic curve. Let $L = V(y - 1, z, w)$ be the line $\{(t, 1, 0, 0)\}$.

Step 1: discovery (*Macaulay2*). *Macaulay2* computes $I + J = (1)$, where $J = (y - 1, z, w)$: the varieties are disjoint. This is the kind of fact a researcher finds in seconds and would previously have had to re-derive by hand to use in a formal development.

Step 2: certificate (*Macaulay2*). The bridge package asks for a membership certificate for 1 and obtains cofactors—in this instance

$$1 = (-1) \cdot (xz - y^2) + (-y - 1) \cdot (y - 1) + x \cdot z,$$

a two-line identity that is trivial to check and encapsulates the entire computation. It also exports a Gröbner certificate for I (three S-pairs with standard representations) and the minimal graded free resolution of R/I , the Eagon–Northcott complex [EN62]

$$0 \rightarrow R(-3)^2 \xrightarrow{d_2} R(-2)^3 \xrightarrow{d_1} R \rightarrow R/I \rightarrow 0,$$

as a graded-complex claim. The full document is 4.8 kB of JSON.

Step 3: verification (*Lean*). `m2lean-check` accepts all four claims and reports their assurance levels:

claim	verdict	level
<code>unit1</code> (membership of 1 in $I+J$)	accepted	proved
<code>gb1</code> (Gröbner basis of I)	accepted	checked
<code>res1</code> (graded complex, Betti data 1, 3, 2)	accepted	checked
<code>cx1</code> ($d_1 d_2 = 0$)	accepted	proved

Step 4: mathematics (*Lean*). The example file re-checks the certificate in the kernel and applies the soundness theorems, yielding (verbatim from the repository):

```

theorem twisted_cubic_cone_misses_line
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra ℚK]
  (p : Fin 4 → K)
  (h1 : aeval p (X 0 * X 2 - X 1 ^ 2) = 0)
  (h2 : aeval p (X 0 * X 3 - X 1 * X 2) = 0)
  (h3 : aeval p (X 1 * X 3 - X 2 ^ 2) = 0)
  (h4 : aeval p (X 1 - 1) = 0)
  (h5 : aeval p (X 2) = 0)
  (h6 : aeval p (X 3) = 0) : False

theorem resolution_composes_to_zero :
  toMatrix 4 1 3 resD1 * toMatrix 4 3 2 resD2 = 0

```

The first statement quantifies over every nontrivial commutative $\mathbb{Q}$ -algebra: no point of any base change lies on both varieties. *Macaulay2* computed; the certificate crossed the boundary; *Lean* supplied semantics and generality. Neither system alone produces this artifact: *Macaulay2* has no notion of “for all $\mathbb{Q}$ -algebras K ,” and *Lean* would not have found the cofactors. An axiom audit confirms that these theorems depend on exactly *Lean*’s three standard axioms (`propext`, `Classical.choice`, `Quot.sound`)—no sorry, no project-specific axioms, no `native_decide`.

8 Adversarial evaluation

The acceptance criterion that matters most for a bridge is not that honest certificates pass but that dishonest ones fail. The test suite (`scripts/run-checks.sh`) exercises both directions; the corrupted certificates are obtained by *mutating real Macaulay2 output*, not by constructing strawmen:

document	defect	outcome
5 valid docs (2 <i>Macaulay2</i> -generated)	—	all accepted
<code>corrupt-membership</code>	wrong cofactor	rejected (verification)
<code>corrupt-gb-quotient</code>	S-pair quotient scaled by 2	rejected (verification)
<code>corrupt-gb-missing-spair</code>	required S-pair deleted	rejected (verification)
<code>corrupt-complex</code>	differential entry altered	rejected (verification)
<code>noncanonical-unsorted</code>	terms misordered for ring order	rejected (structural)
<code>structure-bad-arity</code>	exponent arity mismatch	rejected (structural)
<code>parse-truncated</code>	invalid JSON	rejected (parse)

All checks, including regenerating the *Macaulay2* documents, complete in seconds on a laptop; kernel-checking the flagship certificate adds about three seconds to the *Lean* build. At this scale certificate size is a non-issue (the protocol’s verbose JSON is a development encoding; the specification is deliberately encoding-independent so a compact binary form can be added without semantic change).

9 What each system gains

For *Lean*. Immediately: a disciplined way to import serious computation. The pattern “*Macaulay2* finds the witness, a small checker consumes it” extends well beyond membership—elimination witnesses, Hilbert-series data, syzygies, radical membership via Rabinowitsch, regular sequences—and each new certificate kind is a checker plus a soundness theorem, not a verified reimplementa-tion of a CAS. Less obviously: the checkers themselves generate a formalization agenda ordered by computational relevance. Our Gröbner checker is an exact, executable statement of what remains to be proved (Buchberger’s criterion for standard representations over the two protocol orders); formalizing against a fixed executable target is a better-posed task than formalizing a textbook chapter.

For *Macaulay2*. The bridge is also a mirror. Exporting certificates forced precise answers to questions that are usually implicit: exactly which monomial order convention is in force, what the division algorithm guarantees about its quotients, which results are deterministic. We expect the deeper influence to run through specifications: “`certify gb I`” gives *Macaulay2* users a machine-checkable notion of what `gb` promises, distinct from what the implementation happens to do—and a user-visible difference between “computed” and “certified” creates the incentive for algorithms to expose the evidence they already possess internally (change matrices and reduction traces that engines routinely compute and discard).

10 Limitations and roadmap

The current fragment is exactly what the trust model can carry today, and its edges are documented rather than blurred: coefficients are $\mathbb{Q}$ in the proved semantics (prime fields parse and validate but their claims are not yet checkable; the intended interpretation is $\mathbf{ZMod}\ p$); `ChainComplex` certifies a complex, not exactness—the specification is explicit that $d_i d_{i+1} = 0$ establishes only a complex, and exactness certificates (via certified syzygy claims, or rank and depth data in the style of Buchsbaum–Eisenbud) are the designed next step of the homological tier; Gröbner and graded claims are **checked**, not **proved**; and certificates are eagerly embedded rather than streamed, which will not survive contact with large computations without the planned compact encoding and reflective checkers.

The near-term roadmap, in order of leverage: formalize Buchberger’s criterion against the existing checker (promoting every GB certificate to **proved**); add negative certificates (normal forms witnessing non-membership, which require a proved GB tier to be sound); prime-field semantics; syzygy and exactness claims; a `by macaulay2` tactic wrapping the subprocess round trip; and a `certify` verb inside *Macaulay2* sessions. The long-term test of the project is whether conjectures, computations, certificates, and theorems can move between the two ecosystems while retaining precise meaning and explicit trust—so that a computation performed in 2026 can still be replayed, and its theorem still trusted, by systems that have long since changed.

Acknowledgements

This project’s design and implementation were carried out with the assistance of Claude (Anthropic). We thank the Macaulay2 and *Lean/mathlib* communities, whose public interfaces and libraries made a two-system bridge feasible as a vertical slice.

References

- [BBFF20] Haniel Barbosa, Jasmin Christian Blanchette, Mathias Fleury, and Pascal Fontaine. Scalable fine-grained proofs for formula processing. In *Journal of Automated Reasoning*, volume 64, pages 485–510, 2020.
- [Bha22] Dhruv Bhatia. `polyrith`: a Lean tactic closing polynomial arithmetic goals via an external computer algebra system. `mathlib4`, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Polyrith.html, 2022.
- [Buc70] Bruno Buchberger. Ein algorithmisches Kriterium für die Lösbarkeit eines algebraischen Gleichungssystems. *Aequationes Mathematicae*, 4:374–383, 1970.
- [BW93] Thomas Becker and Volker Weispfenning. *Gröbner Bases: A Computational Approach to Commutative Algebra*, volume 141 of *Graduate Texts in Mathematics*. Springer, 1993.
- [CDM13] Cyril Cohen, Maxime Dénès, and Anders Mörtberg. Refinements for free! In *Certified Programs and Proofs (CPP 2013)*, volume 8307 of *Lecture Notes in Computer Science*, pages 147–162. Springer, 2013.
- [CLO15] David A. Cox, John Little, and Donal O’Shea. *Ideals, Varieties, and Algorithms*. Undergraduate Texts in Mathematics. Springer, 4th edition, 2015.
- [DM05] David Delahaye and Micaela Mayero. Dealing with algebraic expressions over a field in Coq using Maple. In *Journal of Symbolic Computation*, volume 39, pages 569–592, 2005.
- [dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [Eis95] David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*, volume 150 of *Graduate Texts in Mathematics*. Springer, 1995.
- [EN62] J. A. Eagon and D. G. Northcott. Ideals defined by matrices and a certain complex associated with them. *Proceedings of the Royal Society of London. Series A*, 269:188–204, 1962.
- [Gol22] Abby J. Goldberg. `linear_combination`: certifying ring equalities from explicit coefficients. `mathlib4`, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html, 2022.
- [GS24] Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. Available at <http://www2.macaulay2.com>, 2024.

- [Har07] John Harrison. Verifying nonlinear real formulas via sums of squares. *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, LNCS 4732, pages 102–118, 2007.
- [HT98] John Harrison and Laurent Théry. A skeptic’s approach to combining HOL and Maple. *Journal of Automated Reasoning*, 21(3):279–294, 1998.
- [IM16] Fabian Immler and Alexander Maletsky. Gröbner bases theory. *Archive of Formal Proofs*, 2016. https://isa-afp.org/entries/Groebner_Bases.html, formal proof development.
- [KW07] Cezary Kaliszyk and Freek Wiedijk. Certified computer algebra on top of an interactive theorem prover. In *Towards Mechanized Mathematical Assistants (Calculus/MKM 2007)*, volume 4573 of *Lecture Notes in Computer Science*, pages 94–105. Springer, 2007.
- [Taw26] Andrew Tawfeek. M2lean: repository and artifact. <https://github.com/andrew-tawfeek/M2Lean>, 2026.
- [Thé01] Laurent Théry. A machine-checked implementation of Buchberger’s algorithm. *Journal of Automated Reasoning*, 26(2):107–137, 2001.
- [The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

A A complete protocol document

The valid fixture `membership.json`, abridged only in whitespace: the claim certifies $2x \in (x + y, x - y)$ in $\mathbb{Q}[x, y]$ with cofactors $(1, 1)$.

```
{ "m2leanVersion": "0.1.0",
  "documentId": "fixture-membership",
  "objects": [
    { "id": "QQ", "kind": "RationalField" },
    { "id": "R", "kind": "PolynomialRing", "coefficients": "QQ",
      "variables": ["x", "y"], "monomialOrder": { "kind": "GRevLex" } },
    { "id": "I", "kind": "Ideal", "ring": "R", "generators": [
      { "terms": [
        { "coefficient": { "num": "1", "den": "1" }, "exponents": [1, 0] },
        { "coefficient": { "num": "1", "den": "1" }, "exponents": [0, 1] } ] },
      { "terms": [
        { "coefficient": { "num": "1", "den": "1" }, "exponents": [1, 0] },
        { "coefficient": { "num": "-1", "den": "1" }, "exponents": [0, 1] } ] } ] },
    "claims": [
      { "id": "c1", "kind": "IdealMembership", "ideal": "I",
        "element": { "terms": [
          { "coefficient": { "num": "2", "den": "1" }, "exponents": [1, 0] } ] },
        "evidence": { "cofactors": [
          { "terms": [ { "coefficient": { "num": "1", "den": "1" },
            "exponents": [0, 0] } ] },
          { "terms": [ { "coefficient": { "num": "1", "den": "1" },
            "exponents": [0, 0] } ] } ] },
        "provenance": { "producer": "handwritten", "producerVersion": "n/a",
```

```
"packageVersion": "0.1.0", "deterministic": true } }
```

The corresponding verification report:

```
{ "m2leanVersion": "0.1.0",  
  "reportFor": "fixture-membership",  
  "results": [  
    { "claim": "c1", "status": "accepted", "assurance": "proved",  
      "message": "element lies in the ideal" } ],  
  "provenance": { "producer": "m2lean-check (Lean 4)",  
                  "packageVersion": "0.1.0" } }
```